

1. Datos generales

Curso	Inteligencia Artificial Aplicada al Control (ICA636-0001)
Trabajo	Tarea 1 — Redes Neuronales Estáticas
Alumno	Joel Arzapalo Guerra
Código	20032006
Fecha	8 de julio de 2026

2. Introducción

Este trabajo analiza cuatro programas MATLAB que entrenan una **red neuronal estática** (red multicapa de una capa oculta) para **aproximar funciones** a partir de datos con ruido, mediante *retropropagación del error* (descenso de gradiente). Todos comparten la misma arquitectura: entrada $\rightarrow$ capa oculta (nm neuronas con función de activación no lineal) $\rightarrow$ salida lineal, con pesos v (entrada-oculta) y w (oculta-salida) tratados en forma matricial.

Los cuatro scripts ilustran, de forma incremental, los aspectos fundamentales del entrenamiento:

- `NeuronLinealPatron.m`: ajuste de una recta ruidosa con entrenamiento *patrón* (en línea): los pesos se actualizan tras *cada* muestra.
- `NeuronLinealPatronBatch.m`: el mismo problema con entrenamiento por *lote*: el gradiente se acumula sobre todas las muestras y se actualiza una vez por época.
- `NeuronCubicaEscalamiento.m`: ajuste de una función *cúbica* y estudio del **problema de escalamiento** de la salida.
- `NeuronDosEntradasDosSalidas.m`: aproximación de **múltiples entradas y múltiples salidas** (2 entradas y 2 salidas).

Los comentarios de los scripts proponen explícitamente estudiar el número de neuronas nm , el valor inicial de los pesos, el tipo de activación y la neurona de *sesgo* (bias). El informe cubre (i) el **análisis del código** y (ii) los **resultados experimentales** sobre esos parámetros. Los scripts (interactivos, con `input()`) se adaptaron a versiones no interactivas y reproducibles (semilla fija), encapsulando el entrenamiento en la función `lib_mlp.m`, fiel a la formulación original.

3. Análisis del código

3.1. Estructura común

Todos generan datos $\{x_k, y_k^*\}$ (una función objetivo más ruido gaussiano), inicializan los pesos aleatoriamente y ejecutan el bucle de entrenamiento. La salida de cada neurona oculta es $n = f(m)$ con $m = v^\top \text{in}$, y la salida de la red es $\text{out} = w^\top n$. Las activaciones disponibles (y sus derivadas, necesarias para la regla de la cadena) son:

$$\underbrace{n = \frac{1}{1+e^{-m}}}_{\text{logística}}, \underbrace{n = \frac{2}{1+e^{-m}} - 1}_{\text{bipolar}}, \underbrace{n = e^{-m^2}}_{\text{gaussiana}}.$$

El costo es $J = \frac{1}{2N} \sum_k e_k^2$ con $e = \text{out} - y^*$, y los gradientes son $\partial J / \partial w = n e^\top$ y $\partial J / \partial v = \text{in} (f'(m) \odot (w e))^\top$.

3.2. Patrón vs lote

La diferencia esencial entre los dos primeros scripts es *cuándo* se actualizan los pesos:

```
1 for k = 1:N
2     ... er = out - yb(k,1);
3     dJdw = er.*n; dJdv = er.*in*(w.*dndm)';
4     w = w - eta*dJdw; % actualizacion INMEDIATA (cada muestra)
5     v = v - eta*dJdv;
6 end
```

Listing 1: Entrenamiento PATRON: se actualiza dentro del bucle de muestras.

```
1 for k = 1:N
2     ... dJdw = dJdw + n*er'; dJdv = dJdv + in*(dndm.*(w*er))';
3 end
4 w = w - eta*dJdw/N; % actualizacion tras recorrer TODO el conjunto
5 v = v - eta*dJdv/N;
```

Listing 2: Entrenamiento LOTE: se acumula y se actualiza una vez por epoca.

El **sesgo** (bias) se implementa añadiendo una columna constante a la entrada ($x = [x \text{ ones}(N,1)]$, $ne = ne+1$), lo que da a cada neurona un término independiente. El criterio de parada usa la mejora relativa porcentual del costo ($dJpor < errorporc$).

3.3. Escalamiento (cúbica) y salida vectorial (MIMO)

`NeuronCubicaEscalamiento.m` multiplica la salida deseada por un **FACTOR** para estudiar qué pasa cuando su magnitud crece: con tasa de aprendizaje fija, el entrenamiento se vuelve sensible a la escala de y^* . `NeuronDosEntradasDosSalidas.m` generaliza a $ns = 2$ salidas: w pasa a ser $nm \times ns$ y el error e es un vector; la misma regla de la cadena se aplica en bloque. Incluye un factor de pendiente a por neurona en la sigmoide $n = 2/(1 + e^{-m/a}) - 1$.

4. Resultados experimentales

Resultados con las versiones no interactivas (semilla fija). Las figuras se generan con `Informe/exp/run_all.m`.

4.1. Ajuste de una recta: patrón vs lote, bias, η e inicialización

Se ajustó la recta ruidosa $y = x + 2 + 0,2\mathcal{N}$ ($n_m = 10$). Ambos modos de entrenamiento alcanzan el nivel de ruido irreducible ($\sigma = 0,2$): RMSE 0,231 en modo **patrón** y 0,256 en **lote** (Fig. 1, ajuste en Fig. 2). El modo patrón converge en menos épocas —actualiza los pesos N veces por época— pero con una trayectoria de costo más ruidosa; el lote es más suave y estable por época.

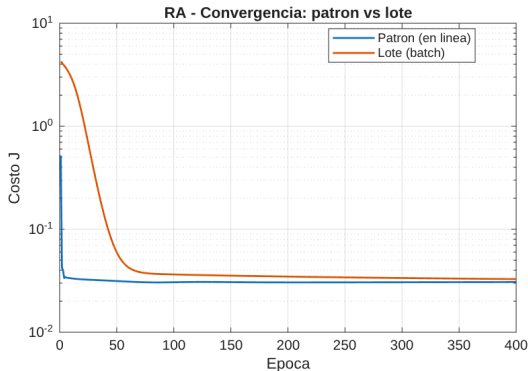


Figura 1: Convergencia: patrón vs lote.

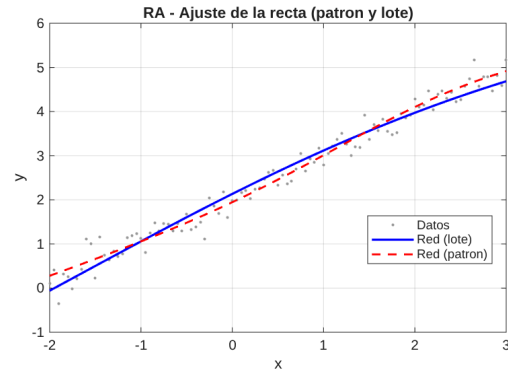


Figura 2: Ajuste de la recta (patrón y lote).

Sesgo. Es el efecto más marcado: *con* neurona de sesgo el RMSE es 0,234, pero *sin* sesgo sube a 1,95 (Fig. 3). Sin el término independiente la red no puede representar el desplazamiento $b = 2$ de la recta (con activación bipolar, en $x = 0$ todas las salidas ocultas valen 0 y la red solo produce 0), por lo que el ajuste falla.

Tasa η e inicialización. La tasa de aprendizaje muestra el compromiso habitual (Fig. 4): η pequeño converge lento y η grande, rápido pero con oscilaciones. La escala inicial de los pesos influye poco en el resultado final (RMSE entre 0,224 y 0,263), aunque una inicialización muy grande (5,0) empeora levemente la convergencia (Fig. 5).

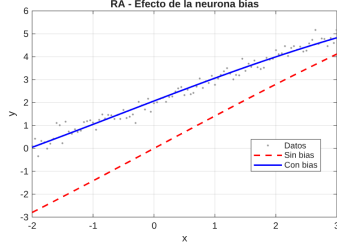


Figura 3: Con y sin sesgo.

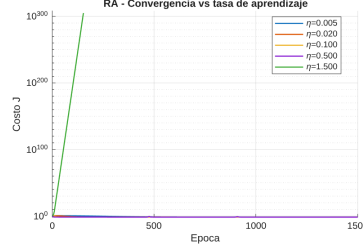


Figura 4: Convergencia vs η .

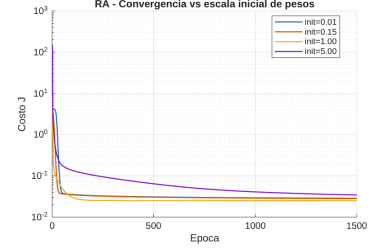


Figura 5: Convergencia vs init.

4.2. Aproximación de una cúbica: neuronas y activación

Se aproximó una función cúbica ruidosa. El **número de neuronas** determina la capacidad de la red (Tabla 1, Fig. 6): con $n_m = 1-2$ la red *subajusta* (RMSE 1,19 y 0,81), a partir de $n_m = 5$ el ajuste alcanza el nivel de ruido (RMSE $\approx 0,26$) y añadir más neuronas no mejora —incluso empeora levemente, por la mayor dificultad de entrenar más parámetros en un número fijo de épocas. La Fig. 7 contrasta el subajuste ($n_m = 2$) con un ajuste adecuado.

Cuadro 1: RMSE del ajuste de la cúbica según el número de neuronas ocultas.

n_m	1	2	5	10	25	50
RMSE	1.19	0.81	0.26	0.33	0.39	0.37

En cuanto a la **función de activación** (Fig. 8), las tres logran ajustar la cúbica; en este problema la gaussiana obtuvo el menor error (RMSE 0,237), seguida de la logística (0,327) y la bipolar (0,395).

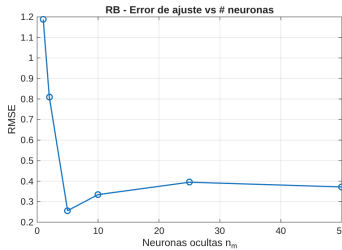


Figura 6: RMSE vs n_m .

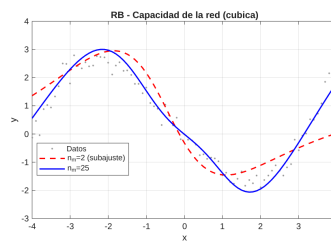


Figura 7: Subajuste vs ajuste.

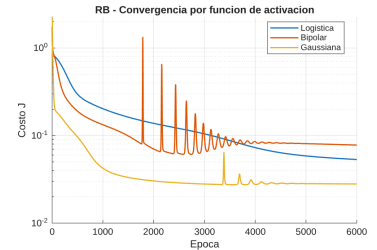


Figura 8: Activación.

4.3. El problema de escalamiento de la salida

Se multiplicó la salida deseada por un **FACTOR** creciente, entrenando con la *misma* tasa $\eta = 0,1$ en dos variantes: sin escalar la salida, y escalándola a $\approx [-1, 1]$ (para luego devolver la predicción a su escala original). El error se mide contra la señal *limpia*. El resultado (Tabla 2, Fig. 9) muestra el problema de escalamiento: **sin escalar, la calidad del ajuste depende fuertemente**

de la magnitud —de un error relativo de 0,05 para magnitudes moderadas hasta 0,63 (ajuste pobre) para magnitudes grandes, con la misma η —. **Escalando la salida el error es constante ($\approx 0,21$) e independiente de la magnitud:** un único η funciona para cualquier escala. La lección práctica es *normalizar la salida* (y las entradas) antes de entrenar, para que la elección de η no dependa de las unidades del problema.

Cuadro 2: Error relativo (vs señal limpia) según la magnitud de la salida y η fijo.

FACTOR	1	10	100	1000	5000
Sin escalar	0.194	0.047	0.295	0.634	0.633
Escalada $[-1, 1]$	0.212	0.212	0.212	0.212	0.212

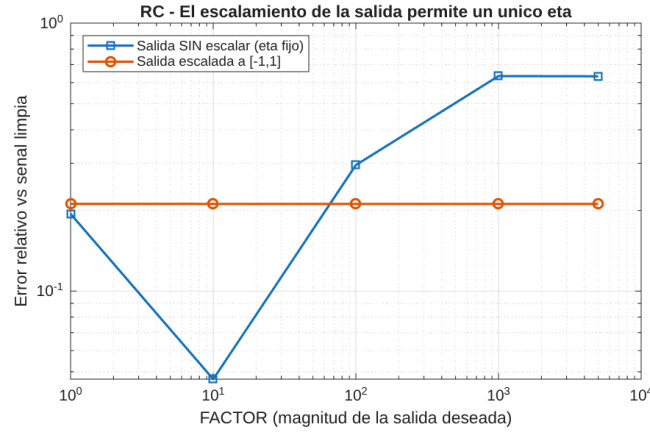


Figura 9: El escalamiento de la salida hace que un único η funcione para toda magnitud (escala log-log).

4.4. Aproximación de múltiples entradas y salidas (2 entradas, 2 salidas)

La misma arquitectura se extiende a dos salidas (w pasa a ser $n_m \times 2$). Con $n_m = 50$ la red aproxima simultáneamente las dos funciones objetivo con RMSE 0,354 para y_1 y 0,116 para y_2 (Fig. 10), del orden del ruido añadido a cada salida. La convergencia (Fig. 11) es monótona. Esto confirma que una única red de una capa oculta puede modelar relaciones de varias entradas y varias salidas.

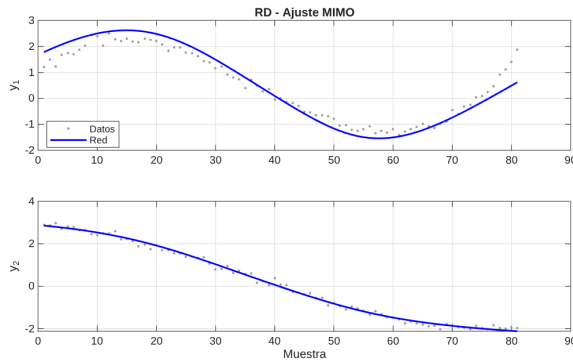


Figura 10: Ajuste de las dos salidas.

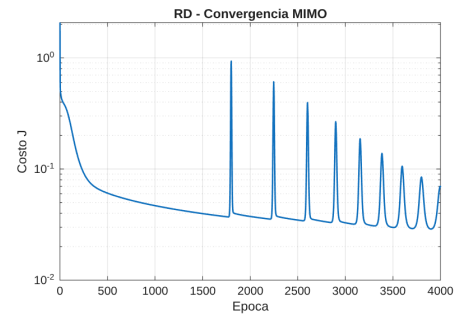


Figura 11: Convergencia (dos salidas).

5. Conclusiones

1. **Patrón vs lote** (Sec. 4.1): ambos ajustan la recta hasta el nivel de ruido; el entrenamiento patrón (en línea) converge en menos épocas pero de forma más ruidosa, mientras que el lote es más estable por época.
2. **La neurona de sesgo es esencial** para representar desplazamientos: sin ella el ajuste de la recta falla (RMSE 1,95 frente a 0,23 con sesgo).
3. **El número de neuronas fija la capacidad** (Sec. 4.2): muy pocas subajustan; a partir de un umbral ($n_m \approx 5$ para la cúbica) se alcanza el nivel de ruido y más neuronas no ayudan (e incluso complican el entrenamiento a épocas fijas).
4. **La activación importa poco si hay capacidad suficiente**: las tres funciones ensayadas ajustan la cúbica, con diferencias menores de error.
5. **El escalamiento es clave** (Sec. 4.3): sin normalizar la salida, el desempeño con una η fija depende fuertemente de la magnitud del problema; escalando la salida (y las entradas) a un rango normalizado, un mismo η funciona para cualquier escala. Junto con la elección de η , es el ajuste práctico más importante.
6. **La arquitectura admite múltiples salidas** (Sec. 4.4): una sola red de una capa oculta aproxima relaciones de varias entradas y varias salidas.

A. Archivos del proyecto

Los experimentos se ejecutan con `Informe/exp/run_all.m`. El entrenamiento está en `lib_mlp.m` (una capa oculta, retropropagación, con modos patrón/lote y activaciones logística/bipolar/-gaussiana). Cada experimento RA–RD guarda sus figuras en `Informe/figs/` y sus métricas en `*_metrics.mat`.